Fast Actual/Expected Data Processing

Introduction

Python is a good programming language choice for the medical director/data scientist who needs to parse text from lab reports, paramedical exams, and underwriting notes. However, Python is a general programming language and can do much more. When combined with a third-party module named Polars, it can rapidly translate large datasets of mortality experience into data suitable for pivot table analysis.

The Python interpreter and its extensive standard library are freely available for all major platforms.[i] Polars, a self-described "a blazingly fast DataFrame library for manipulating structured data", is available for the Python, R, NodeJS, and Rust programming languages.[ii]

In this context, scripts are text files consisting of sequential programming statements that are accumulated by an iterative process: edit -> run -> evaluated -> repeat. As an interpreted language, Python is well-suited for this cyclical process. Each time it is run, Python reads the script file afresh before rapidly translating the instructions into action. This provides the developer with a fast turn-around while testing ideas. However, programs run by interpreters are inherently slower than compiled programs that are run directly from binary.

Often, scripts reflect the steps that one would take if one were doing the processing "by hand" in MS Excel where one tends to work with the data row by row. A less intuitive column-oriented approach is generally faster, and in this case it is required to take advantage of the speed available in Polars.. A complete Polars/Python script file with annotations is available in Appendix 1, while the critical concepts are discussed in the implementation section. See Appendix 2 for a description of the 24,528 row Health Risk Assessment experience dataset as well as information about the HMD mortality table used to provide *qx*'s.

The script in Appendix 1 is blazingly fast. Similar programs written in other languages take minutes to process the same example dataset. This Polars/Python script does it in 0.15 seconds on a Dell All-In-One with an Intel i7-1355U (1.70 GHz) processor and 32 GB of RAM.

Motivation

The first 15 rows of a dataframe holding synthetic HRA data are shown in Table 1. Dataframes are commonly used 2-dimensional structures with the following characteristics:

- Each column is a variable – *single, measurable characteristic (age, weight, location, etc.)*
- Each row is an observation – *measurements for a single observation unit*
- Each cell is a single value – *the value of one measurement for one observation*
- Each table contains observations of single type.

Table 1

UID	examD	exitD	vs	age	dob	sex	dxGrp	examY	site	ageBand	mr
10001	1998-01-02	2004-04-28	dead	71	1926-03-17	M	diabetes	1998	AL	70 - 74	3.5
10002	1998-01-05	2005-12-11	dead	79	1918-05-21	F	cardiac	1998	TX	75 - 79	4.0
10003	1998-01-06	2001-03-20	dead	72	1925-02-21	M	pulm	1998	AZ	70 - 74	5.0
10004	1998-06-10	2005-09-23	dead	75	1922-11-12	M	cardiac	1998	SC	75 - 79	4.0
10005	1998-01-09	2003-12-28	dead	83	1914-10-03	M	other	1998	GA	80 - 84	5.0
10006	1998-01-09	2007-05-31	dead	84	1913-05-23	F	healthy	1998	SC	80 - 84	1.0
10007	1998-01-16	2005-07-08	dead	81	1916-01-29	M	pulm	1998	TX	80 - 84	7.0
10011	1998-01-22	2007-01-10	dead	77	1920-03-14	F	cardiac	1998	GA	75 - 79	4.0
10015	1998-01-26	2002-01-07	dead	75	1923-01-16	M	other	1998	TX	75 - 79	3.5
10016	1998-01-27	2008-06-30	alive	80	1917-10-25	F	healthy	1998	VA	80 - 84	1.5
10017	1998-01-29	2008-06-30	alive	83	1914-05-14	M	cancer	1998	SC	80 - 84	2.5
10018	1998-01-30	2004-11-28	dead	81	1916-02-10	F	other	1998	TX	80 - 84	4.0
10021	1998-02-12	1998-08-24	dead	76	1921-03-09	M	cardiac	1998	VA	75 - 79	4.0
10022	1998-02-20	2004-01-05	dead	74	1923-11-21	M	neuro	1998	SC	70 - 74	5.0
10023	1998-02-24	2008-06-30	alive	75	1922-04-02	F	healthy	1998	AZ	75 - 79	1.25

See Appendix 2 for more details about the dataset and its columns.

Note that every individual's exposure is clearly demarcated in calendar time. However, what is needed for mortality analysis is a dataset where each subject enters at time zero on a study time calendar and study time is demarcated by yearly intervals, as in Table 2.

For example, the experience of the three UID's at the bottom of Table 1: 10021, 10022, and 10023, can be reorganized as shown in Table 2.

Table 2

UID	intvl	intvl_date	age	sex	mr	dxGrp	site	ageBand	year	persYrs	actual	qx	expected
10021	1	2/12/1998	76	M	4.06	cardiac	VA	75 - 79	1998	0.5311	1	0.0524	0.1130
10022	1	2/20/1998	74	M	4.62	neuro	SC	70 - 74	1998	1	0	0.0442	0.2043
10022	2	2/20/1999	75	M	4.62	neuro	SC	70 - 74	1999	1	0	0.0477	0.2203
10022	3	2/20/2000	76	M	4.62	neuro	SC	70 - 74	2000	1	0	0.0505	0.2331
10022	4	2/20/2001	77	M	4.62	neuro	SC	70 - 74	2001	1	0	0.0543	0.2507
10022	5	2/20/2002	78	M	4.62	neuro	SC	70 - 74	2002	1	0	0.0582	0.2690
10022	6	2/20/2003	79	M	4.62	neuro	SC	70 - 74	2003	0.8761	1	0.0626	0.2534
10023	1	2/24/1998	75	F	1.33	healthy	AZ	75 - 79	1998	1	0	0.0298	0.0396
10023	2	2/24/1999	76	F	1.33	healthy	AZ	75 - 79	1999	1	0	0.0330	0.0439
10023	3	2/24/2000	77	F	1.33	healthy	AZ	75 - 79	2000	1	0	0.0363	0.0483
10023	4	2/24/2001	78	F	1.33	healthy	AZ	75 - 79	2001	1	0	0.0397	0.0527
10023	5	2/24/2002	79	F	1.33	healthy	AZ	75 - 79	2002	1	0	0.0434	0.0577
10023	6	2/24/2003	80	F	1.33	healthy	AZ	75 - 79	2003	1	0	0.0497	0.0662
10023	7	2/24/2004	81	F	1.33	healthy	AZ	75 - 79	2004	1	0	0.0534	0.0710
10023	8	2/24/2005	82	F	1.33	healthy	AZ	75 - 79	2005	1	0	0.0591	0.0786
10023	9	2/24/2006	83	F	1.33	healthy	AZ	75 - 79	2006	1	0	0.0636	0.0845
10023	10	2/24/2007	84	F	1.33	healthy	AZ	75 - 79	2007	1	0	0.0696	0.0926
10023	11	2/24/2008	85	F	1.33	healthy	AZ	75 - 79	2008	0.3504	0	0.0789	0.0368

Table 2 is actual output from the Polars/Python script. Both script and output are discussed in the next sections.

Implementation

Mortality Methodology

The 'intvl_date is the first day of exposure for that interval and the interval extends either through the day prior to the next 'intvl_date' or through the 'exitD' for a subject's last interval. Exposure is defined as one year for any interval that a subject survives. For “last” intervals where subject either dies or is withdrawn, exposure is the days spent in interval, inclusive of 'intvl_date' and 'exitD' divided by 365.25.

Where a death occurs on an 'intvl_date', the actual is assigned to the 'intvl_date' interval and that interval will have 1 day of exposure. In the case of withdrawal on an 'intvl_date', the interval will also have 1 day of exposure.

These parameters are easily modified in the script to accommodate studies with incomplete information, or dollar measured mortality, or for consistency with other analyses.

Column Manipulation

In line 2, polars is imported “as pl”. Any expression in the script that starts with “pl” is a polars object or function. A column can simply be referred to by its name or position, e.g. 'exitD' or [:2], but when the expression pl.col('exitD') is used, it becomes a polars object and it can access routines in the polars pre-compiled library.

Line 44 is the first calculation in this script. The entire 'examD' column of 24,528 dates is subtracted from the entire 'exitD' column of 24,528 dates; the interim result is divided by 365.25 and the results are stored in a new column 'totalIntvls' of 24,528 integers. The arithmetic is performed by matrix methods and the creation of 'totalIntvls' is fast.

In line 48 a temporary column, 'idx_list', is created in a new dataframe, df_indices, to hold identical lists of interval indices. A temporary column is necessary to allow the calculations in line 52 to be done as fast column manipulations.

Row Expansion

Row expansion is essential for the process. Basically, there needs to be a row for every interval where a study subject is exposed to risk. Row expansion methods are available in other data management programs such as Stata. Polars is different in that expansion is done in two steps:

1. The Dataframe method .repeat_by() repeats the contents of target cells the number of specified times. Each target cell then contains a list of repeats of its original value.
2. The Dataframe method .explode() converts the cell lists into repeated rows

This two step expansion allows one to slip in the appropriate interval numbers. First, a df_indices dataframe is created from two df columns, 'UID' and 'totalIntvls' along with a temporary column 'idx_list' that stores identical lists of the numbers 1 through 11 that will be used to label intervals (line 48). A more permanent column of lists, 'intvl' is created by slicing the lists in 'idx_list', where the length of each slice is the number in 'totalIntvls' (line 52).

Table 3

UID --- list[i64]	intvl --- list[i64]
[10001, 10001, … 10001]	[1, 2, … 7]
[10002, 10002, … 10002]	[1, 2, … 8]
[10003, 10003, … 10003]	[1, 2, … 4]
[10004, 10004, … 10004]	[1, 2, … 8]
[10005, 10005, … 10005]	[1, 2, … 6]
[10006, 10006, … 10006]	[1, 2, … 10]
[10007, 10007, … 10007]	[1, 2, … 8]
[10011, 10011, … 10011]	[1, 2, … 9]
[10015, 10015, … 10015]	[1, 2, … 4]
[10016, 10016, … 10016]	[1, 2, … 11]

This new column, 'intvl', needs to be joined to the main dataframe df prior to the explode() step. The df('UID') column will contain lists of UID's that were repeated 'totalIntvls' times. For a successful join, the df_indices('UID') column values need to match exactly (lines 56-57).

All the columns in df, the main dataframe, are then repeated the number of times in 'totalIntvls'. After the join with df_indices, all the lists are "exploded" in line 6, resulting in the following dataframe:

Table 4

UID --- i64	vs --- enum	examD --- date	exitD --- date	age --- i64	year --- i64	totInvls --- i32	intvl --- i64
10001	dead	1998-01-02	2004-04-28	71	1998	7	1
10001	dead	1998-01-02	2004-04-28	71	1998	7	2
10001	dead	1998-01-02	2004-04-28	71	1998	7	3
10001	dead	1998-01-02	2004-04-28	71	1998	7	4
10001	dead	1998-01-02	2004-04-28	71	1998	7	5
10001	dead	1998-01-02	2004-04-28	71	1998	7	6
10001	dead	1998-01-02	2004-04-28	71	1998	7	7
10002	dead	1998-01-05	2005-12-11	79	1998	8	1
10002	dead	1998-01-05	2005-12-11	79	1998	8	2
10002	dead	1998-01-05	2005-12-11	79	1998	8	3

The balance of the Python code consists of straightforward steps:

1. Calculation of interval start dates.
2. Creation of columns for interval year, exposure (person-years), and actual deaths.
3. A join with df_details to bring 'site', 'ageBand', etc. back into the main df.
4. A join with df_mort to get a *qx* for each interval year, interval age, and sex.
5. Calculation of expected deaths for each interval.
6. Saving the results as a comma separated value file.

Big Data

To further test the speed of the Polars/Python script, a mega-dataset consisting of 10 lightly modified versions of the example study dataset. The 245,280 rows were stored as a 21 MB csv file. This was the source for the Polars/Python script. Similarly, a 10-fold larger dataset of 2,452,800 rows was created and stored as a 213.5 MB csv file.

Results

Table 5

UID	age	sex	mr	dxGrp	site	ageBand	intvl	intvl_date	year	persYrs	actual	qx	expected
80833	74	M	1.89	diabetes	GA	70 - 74	1	2/14/2002	2002	1	0	0.04113	0.0777
80833	75	M	1.89	diabetes	GA	70 - 74	2	2/14/2003	2003	1	0	0.04445	0.0840
80833	76	M	1.89	diabetes	GA	70 - 74	3	2/14/2004	2004	1	0	0.04548	0.0860
80833	77	M	1.89	diabetes	GA	70 - 74	4	2/14/2005	2005	1	0	0.04986	0.0942
80833	78	M	1.89	diabetes	GA	70 - 74	5	2/14/2006	2006	1	0	0.05202	0.0983
80833	79	M	1.89	diabetes	GA	70 - 74	6	2/14/2007	2007	1	0	0.05558	0.1050
80833	80	M	1.89	diabetes	GA	70 - 74	7	2/14/2008	2008	0.377823	0	0.06343	0.0453
80834	69	M	2.66	cardiac	AZ	65 - 69	1	2/15/2002	2002	1	0	0.02631	0.0700
80834	70	M	2.66	cardiac	AZ	65 - 69	2	2/15/2003	2003	1	0	0.02799	0.0745
80834	71	M	2.66	cardiac	AZ	65 - 69	3	2/15/2004	2004	1	0	0.02888	0.0768
80834	72	M	2.66	cardiac	AZ	65 - 69	4	2/15/2005	2005	0.555784	1	0.03140	0.0464
80835	77	M	1.47	healthy	CA	75 - 79	1	2/18/2002	2002	1	0	0.05349	0.0786
80835	78	M	1.47	healthy	CA	75 - 79	2	2/18/2003	2003	1	0	0.05682	0.0835
80835	79	M	1.47	healthy	CA	75 - 79	3	2/18/2004	2004	1	0	0.05971	0.0878
80835	80	M	1.47	healthy	CA	75 - 79	4	2/18/2005	2005	1	0	0.06845	0.1006
80835	81	M	1.47	healthy	CA	75 - 79	5	2/18/2006	2006	1	0	0.07365	0.1083
80835	82	M	1.47	healthy	CA	75 - 79	6	2/18/2007	2007	1	0	0.07833	0.1151
80835	83	M	1.47	healthy	CA	75 - 79	7	2/18/2008	2008	0.366872	0	0.08653	0.0467

The final dataframe consists of 92,696 rows and 14 columns as shown in the above sample.

The final step of writing the dataframe to a csv file makes it easy to load the data in MS Excel for pivot table analysis. Excel can load datasets as large as slightly more than 1 million rows. For any larger dataset, one can use aggregate calculations from polars.

Table 3 shows A/E ratios by age band and sex as well as totals. For the purpose of evaluating the script file, we are glad to see the 1,561 female and 3,894 male actual deaths. Comparison with our raw dataset shows that no deaths have been misplaced.

Table 6

	Females			Males			Totals		
Age Band	actual	expected	A/E	actual	expected	A/E	actual	expected	A/E
65 – 69	56	81.0	0.69	414	549.2	0.75	470	630.2	0.75
70 – 74	243	265.0	0.92	891	1,137.3	0.78	1,134	1,402.3	0.81
75 – 79	596	635.9	0.94	1,346	1,853.2	0.73	1,942	2,489.1	0.78
80 – 84	666	884.5	0.75	1,243	1,908.8	0.65	1,909	2,793.3	0.68
Total	1,561	1,866.4	0.84	3,894	5,448.6	0.71	5,455	7,315.0	0.75

The following Table 7 is truncated to fit the page, but the counts for the missing age bands do appear in the totals.

Table 7

	75 – 79				80 – 84				Totals (including 65 – 74)			
Interval	Entrants	Actual	Expected	A/E	Entrants	Actual	Expected	A/E	Entrants	Actual	Expected	A/E
1	8,129	279	556.2	0.50	5,786	295	652.4	0.45	24,528	812	1,653.6	0.49
2	6,913	339	468.5	0.72	4,915	353	541.8	0.65	21,049	973	1,375.5	0.71
3	5,051	370	388.8	0.95	3,641	327	450.4	0.73	15,227	995	1,138.7	0.87
4	3,682	293	325.8	0.90	2,735	307	374.5	0.82	11,191	844	950.1	0.89
5	2,708	243	266.5	0.91	2,028	240	296.8	0.81	8,138	663	768.8	0.86
6	1,830	153	201.0	0.76	1,313	172	209.3	0.82	5,607	466	577.9	0.81
7	1,112	116	132.5	0.88	769	109	130.3	0.84	3,539	328	387.4	0.85
8	563	76	79.0	0.96	370	50	75.2	0.67	1,904	181	239.3	0.76
9	264	42	46.9	0.90	170	35	42.8	0.82	918	114	146.3	0.78
10	124	27	21.7	1.25	78	20	18.0	1.11	478	72	70.2	1.03
11	32	4	2.2	1.81	19	1	1.9	0.54	116	7	7.1	0.99
Total		1,942	2,489.1	0.78		1,909	2,793.3	0.68		5,455	7,315.0	0.75

Big Data

Running the Polars/Python script on the 245,280 row mega-dataset took 0.49 seconds. The 927,270 row dataframe of results was stored as an 87 MB csv file. MS Excel was able to load the csv file and pivot table analysis was performed without any slowdown. The pivot results were the same as for the 24,528 example dataset, but with 10-fold increases in numbers.

On the 2,452,800 row test file, the Polars/Python script completed its tasks in 5.14 seconds, and the results were stored in an 889.8 MB csv file. This is too big to be loaded into MS Excel, so pivot analysis was performed in Polars, see Table 8. Comparison with Table 6 shows that Table 8 results are 100-fold greater, as one would expect.

Table 8

ageBand	actual_M	actual_F	expected_M	expected_F
65 – 69	41,400	5,600	55,220.8	8,082.5
70 – 74	89,100	24,300	113,853.6	26,460.1
75 – 79	134,600	59,600	185,918.5	63,343.8
80 – 84	124,300	66,600	190,773.6	88,158.7

Discussion

There are other ways to accomplish the same results as this Polars/Python script, but none are nearly as fast. Is speed really important? When one is working with data, one may spend most of one's time in an iterative process of edit -> run -> evaluated -> repeat. If the "run" part of the cycle takes more than a few seconds, the process may become unbearably tedious. The intended study may never be done.

The script was first written in Stata over a decade ago. The Stata script required as much as 3 minutes to process similar datasets. More recently, it was re-written in Python, but without Polars. This script took as much as 5 minutes to run the synthetic data featured in this article. Now that Polars expressions have replaced the entire script, it is able to process the dataset in less than 0.15 seconds.

Column-oriented expressions that use matrix math to rapidly calculate new columns will always be faster than row by row processing. Better yet would be to use Polars objects that have a parallel execution engine, efficient algorithms, and vectorization with SIMD (Single Instruction, Multiple Data).

MS Excel remains the easiest way to perform pivot table analyses, as long as the data fits within Excel's 1,048,576 row and 16,384 column maximums.[iii] Fortunately, Polars can perform basic pivots on any size dataset – even datasets that are too big to fit in a computer's RAM.

Appendix 1 – Polars/Python Script

```python
import os                                   # basic file handling functions
import polars as pl                         # polars, a DataFrame library for manipulating
structured data
from datetime import timedelta              # a single datatype needed for calculations
import US_MortalityTable_03 as USMort

# the working directory
os.chdir(r"C:\Users\dowes\OneDrive\Projects\A2E_paper")

# read the data file into a polars dataframe ('pl' is shorthand for polars)
df = pl.read_csv(r'data/HRA_data.csv', try_parse_dates = True)

# define Enumeration datatypes
enum_vs   = pl.Enum(["alive", "dead"])
enum_sex  = pl.Enum(["M", "F"])
enum_dx   = pl.Enum(["healthy", "cancer", "cardiac", "diabetes", "neuro", "pulm",
       "renal", "other"])
enum_site = pl.Enum(["AL", "AZ", "CA", "FL", "GA", "LA", "SC", "TX", "VA"])
enum_agebnd  = pl.Enum(["65 - 69", "70 - 74", "75 - 79", "80 - 84"])

# cast column values to defined datatypes
df = df.with_columns(
    pl.col("vs").cast(enum_vs),
    pl.col("sex").cast(enum_sex),
    pl.col("dxGrp").cast(enum_dx),
    pl.col("site").cast(enum_site),
    pl.col("ageBand").cast(enum_agebnd))

# rename exam year column
df = df.rename({'examY': 'year'})

# split off columns of "details"
df_details = df.select('UID', 'sex', 'mr', 'dxGrp', 'site', 'ageBand')

# keep columns needed for expansion
df = df.select('UID', 'examD', 'exitD', 'vs', 'age', 'year')

# tDelta constants to be used in calculation
tDelta_1yr = timedelta(days=365, hours=6)
tDelta_1day  = timedelta(days=1)


# calculate total number of intervals for each subject
df = df.with_columns(totalIntvls = ((pl.col('exitD') - pl.col('examD') +
       tDelta_1day)/tDelta_1yr).ceil().cast(pl.Int32))

# create temporary column of identical interval index lists
df_indices = df['UID', 'totalIntvls'].with_columns(idx_list = [1, 2, 3, 4, 5, 6, 7, 8,
       9, 10, 11])

# take slice of idx_list and store list in new column 'intvl'
df_indices = df_indices.with_columns(intvl = pl.col('idx_list').list.slice(0,
       pl.col('totalIntvls')))

# update 'UID' column to lists of repeated UID's
df_indices = df_indices.with_columns(pl.col('UID').repeat_by(pl.col('totalIntvls')))
df_indices = df_indices.select('UID', 'intvl')

# repeat all df columns the number of times in total intervals
df = df.with_columns(pl.all().repeat_by(pl.col('totalIntvls')))

# join to get column of intvl label lists, 'intvl'
df = df.join(df_indices, on='UID', how='left')

```

Appendix 1 – Polars/Python Script

```python
# “explode” lists to get desired tabular output
df = df.explode(pl.all())

# create temporary column of offset years, then calculate 'intvl_date'
df = df.with_columns(offsetYrs = (pl.col("intvl") - 1).cast(pl.String))
df = df.with_columns(intvl_date = pl.col('examD').dt.offset_by((pl.col('offsetYrs')) +
       "y"), intvl_age = pl.col('age') + pl.col('intvl') - 1)

# three important calculations: interval year, exposure, and actual
df = df.with_columns(
    year = pl.col('intvl_date').dt.year(),
    persYrs = pl
        .when((pl.col('intvl_date').dt.offset_by('1y')) < pl.col('exitD'))
        .then(pl.lit(1))
        .otherwise((pl.col('exitD') - pl.col('intvl_date') + tDelta_1day) / tDelta_1yr),
    actual = pl
        .when((pl.col('vs') == "dead") & (pl.col('intvl') == pl.col('totalIntvls')))
        .then(pl.lit(1))
        .otherwise(pl.lit(0)))

# clean up and get ready for joins
df = df.select('UID', 'vs', 'intvl', 'intvl_date', 'intvl_age',
                        'year', 'persYrs', 'actual')
df = df.rename({'intvl_age': 'age'})

# join to bring back “details” like site, age band, etc.
df = df.join(df_details, on='UID', how='left')

# load mortality dataframe
df_mort = USMort.both_sexes_1933_2023()

# join to get qx
df = df.join(df_mort, on=('year','age','sex'), how='left')

# one last calculation, expected deaths
df = df.with_columns(expected = pl.col('persYrs') * pl.col('qx') *
       pl.col('mr')).sort('UID', 'intvl')

# prepare results dataframe for writing to a csv file
df = df.select('UID', 'age', 'sex', 'mr', 'dxGrp', 'site', 'ageBand', 'intvl',
       'intvl_date', 'year', 'persYrs', 'actual', 'qx', 'expected').sort('UID', 'intvl')

df.write_csv(r'data/a2e_results_' + itr + '.csv')
# double-clicking the csv file will open it in MS Excel
```

Appendix 2 – Example Dataset and HMD Mortality Table

Study Dataset

Each row of the 24,528 rows in the file represents a subject's "Health Risk Appraisal" done at one of 9 different sites starting in January 1998 with a study end date of 6/30/2008. Since this is a synthetic dataset, mortality follow-up is 100% complete and error-free.

The fields (columns) are as follows:

- **'UID'** is a unique identifier for each individual
- **'examD'** the date of examination (first exam was 1/2/1998)
- **'exitD'** either the date of death or the study end date (6/30/2008)
- **'vs'** the vital status : ["alive", "dead"]
- **'age'** age at time of exam: 65 – 84
- **'dob'** the date of birth for the individual
- **'sex'** : 'F', 'M'
- **'dxGrp'** will be one of eight categories:
  ["healthy", "cancer", "cardiac", "diabetes", "neuro", "pulm", "renal", "other"]
- **'examY'** the year of examination: 1998 - 2008
- **'site'** will be one of nine exam sites:
  ["AL", "AZ", "CA", "FL", "GA", "LA", "SC", "TX", "VA"]
- **'mr'** the mortality ratio assigned by the examiner
- **'ageBand'** age at the time of exam: ["65 - 69", "70 - 74", "75 - 79", "80 - 84"]

A random ten row sample of the study data is displayed in the dataframe below (split to fit page).

UID	examD	exitD	vs	age	dob	sex
---	---	---	---	---	---	---
i64	date	date	str	i64	date	str
11836	2005-04-29	2008-06-30	alive	72	1932-08-12	F
12406	2006-08-08	2006-12-18	dead	76	1930-03-27	M
30727	2001-08-08	2008-03-13	dead	66	1935-05-14	F
32122	2004-11-22	2008-03-31	dead	80	1924-05-17	F
32391	2005-06-07	2008-06-30	alive	81	1924-05-10	M
32525	2005-09-02	2008-06-30	alive	77	1928-07-26	F
32598	2005-10-14	2008-06-30	alive	79	1926-10-02	F
33086	2006-08-01	2008-06-30	alive	79	1926-09-07	F
33816	2007-06-15	2008-06-30	alive	72	1935-01-20	M
40672	2001-10-12	2008-06-30	alive	65	1936-07-29	M

UID	dxGrp	examY	site	mr	ageBand
---	---	---	---	---	---
i64	str	i64	str	f64	str
11836	healthy	2005	TX	0.98	70 - 74
12406	cardiac	2006	TX	2.66	75 - 79
30727	healthy	2001	CA	1.26	65 - 69
32122	healthy	2004	CA	1.26	80 - 84
32391	healthy	2005	GA	1.26	80 - 84
32525	healthy	2005	TX	1.26	75 - 79
32598	healthy	2005	FL	1.26	75 - 79
33086	healthy	2006	AZ	0.98	75 - 79
33816	healthy	2007	AL	1.26	70 - 74
40672	cardiac	2001	FL	3.71	65 - 69

Appendix 2 – Example Dataset and HMD Mortality Table

Mortality Data

USA mortality tables for both males and females were downloaded from the Human Mortality Database and then stacked. [iv]

The fields (columns) are as follows:

- **'year'** : 1933 – 2023
- **'age'** : 0 – 110
- **'sex'** : 'F', 'M'
- **'qx'** : mortality rate

The following is an excerpt from the mortality dataframe:

year	age	sex	qx
---	---	---	---
i64	i64	enum	f64
1933	0	F	0.05208
1933	0	M	0.06486
1933	1	F	0.00883
1933	1	M	0.00999
1933	2	F	0.00402
…	…	…	…
2023	108	M	0.47703
2023	109	F	0.49014
2023	109	M	0.49286
2023	110	F	1.0
2023	110	M	1.0

[i] Python version 3.14. Python Software Foundation. https://www.python.org/
[ii] Polars BV, De Boelenlaan 7, 1083 HJ, Amsterdam. https://pola.rs/
[iii] Excel specifications and limits. https://support.microsoft.com/en-us/office/excel-specifications-and-limits
[iv] HMD. Human Mortality Database. Max Planck Institute for Demographic Research (Germany), University of California, Berkeley (USA), and French Institute for Demographic Studies (France). Available at www.mortality.org (data downloaded on 24Sep2025, DOI: 10.4054/HMD.Countries).